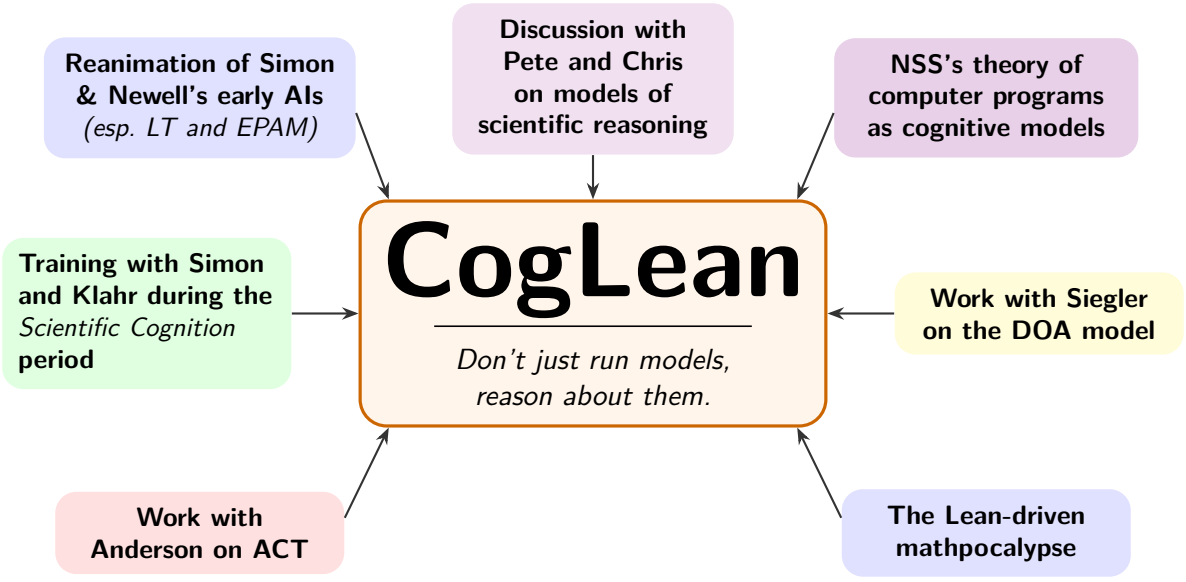
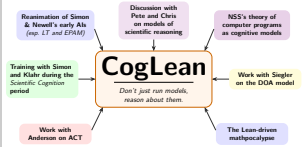
Epoofany

Epoofany

Epoofany.


Reanimation of Simon & Newell's early AIs (esp. LT and EPAM)
Discussion with Pete and Chris on models of scientific reasoning
NSS's theory of computer programs as cognitive models
Training with Simon and Klahr during the Scientific Cognition period
CogLean
Don't just run models, reason about them.
Work with Siegler on the DOA model
Work with Anderson on ACT
The Lean-driven mathpocalypse

Composing Cognitive Theories in CogLean



Where CogLean came from.

Composing Cognitive Theories in CogLean

Multi-Digit Addition

Jeff Shrager

November 2026

Draft for Comment

Composing Cognitive Theories in CogLean

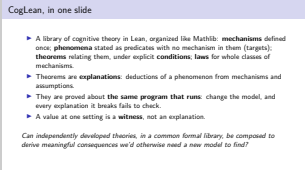


This talk goes with the paper of the same name, a draft for comment; constructive feedback is welcome. A companion talk introduces CogLean, a library of cognitive theory in the Lean proof assistant, and its view that theorems are explanations, proved about the same programs that run. This one is the first extended test: whether independently formalized theories, composed in such a library, give meaningful consequences that we'd otherwise need a whole new model to find. I'll tell it through a hypothetical modeler, Chris, and the central example is multi-digit addition. It's a demonstration, not a new theory of arithmetic.

CogLean, in one slide

- A library of cognitive theory in Lean, organized like Mathlib: **mechanisms** defined once; **phenomena** stated as predicates with no mechanism in them (targets); **theorems** relating them, under explicit **conditions**; **laws** for whole classes of mechanisms.
- Theorems are **explanations**: deductions of a phenomenon from mechanisms and assumptions.
- They are proved about **the same program that runs**: change the model, and every explanation it breaks fails to check.
- A value at one setting is a **witness**, not an explanation.

Can independently developed theories, in a common formal library, be composed to derive meaningful consequences we'd otherwise need a new model to find?



Here's CogLean in one slide; the companion talk has the full version. It's a library of cognitive theory in Lean, organized the way Mathlib organizes mathematics. Mechanisms are defined once. Phenomena are stated as predicates with no mechanism in them: targets. Theorems relate the two, under explicit conditions, and laws let a theorem cover a whole class of mechanisms. The theorems are explanations, deductions of a phenomenon from mechanisms and assumptions, and they're proved about the same definitions that run as the model, so when the model changes, every explanation the change breaks fails to check. A number computed at one setting is a witness that the conditions can be met, not an explanation. The question for this talk: can independently developed theories, in such a library, be composed to give meaningful consequences that we'd otherwise need a whole new model to find?

Chris's question

Chris, a hypothetical modeler who studies children's arithmetic, works with **Robin**, a chatbot colleague: good at finding library results and suggesting proofs, *not* automatically right.

- Children sometimes confidently answer $150 + 100$ with 200. Does that follow from existing mechanisms, and under what conditions?
- Answering it means **composing** two independently formalized theories with a problem solver, and **reusing their theorems**.

Then: what's under the hood of the composition, and what it does and doesn't show.

Chris's question

Chris, a hypothetical modeler who studies children's arithmetic, works with **Robin**, a chatbot colleague: good at finding library results and suggesting proofs, *not* automatically right.

- Children sometimes confidently answer 150 + 100 with 200. Does that follow from existing mechanisms, and under what conditions?
- Answering it means **composing** two independently formalized theories with a problem solver, and **reusing their theorems**.

Then: what's under the hood of the composition, and what it does and doesn't show.

Chris is a hypothetical cognitive scientist who studies how children learn arithmetic, and works with a chatbot colleague, Robin, who is good at finding results in the library and suggesting proofs, but is not automatically right. Chris's question: children sometimes confidently answer one fifty plus one hundred with two hundred. Does that follow from mechanisms we already have, and under exactly what conditions? Answering it means composing two independently formalized theories with a problem solver, and reusing their theorems. After that I'll open the hood on the composition, and finish with what it does and doesn't show.

Part 1

From an observation to a theorem

Does confident error follow from mechanisms we already have?

Part one. Chris's first question, going forward: does a familiar phenomenon, confident errors, follow from mechanisms already in the library?

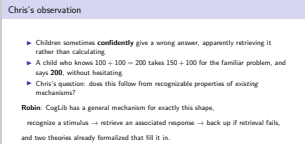
Chris's observation

- Children sometimes **confidently** give a wrong answer, apparently retrieving it rather than calculating.
- A child who knows $100 + 100 = 200$ takes $150 + 100$ for the familiar problem, and says **200**, without hesitating.
- Chris's question: does this follow from recognizable properties of *existing* mechanisms?

Robin: CogLib has a general mechanism for exactly this shape,

recognize a stimulus $\rightarrow$ retrieve an associated response $\rightarrow$ back up if retrieval fails,

and two theories already formalized that fill it in.



Here's Chris's observation. Children sometimes give a wrong answer confidently: they seem to be retrieving it, not calculating it. A child who knows one hundred plus one hundred is two hundred takes one fifty plus one hundred for the familiar problem, and says two hundred without hesitating. Chris's question is whether that follows from recognizable properties of mechanisms we already have, rather than from something invented for the purpose. Robin points out that the library has a general mechanism of exactly this shape: recognize a stimulus, retrieve a response associated with what was recognized, and fall back on a backup process if retrieval fails. And two existing theories, already formalized on their own, fill it in.

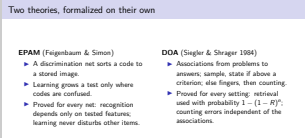
Two theories, formalized on their own

EPAM (Feigenbaum & Simon)

- A discrimination net sorts a code to a stored image.
- Learning grows a test only where codes are confused.
- Proved for every net: recognition depends only on tested features; learning never disturbs other items.

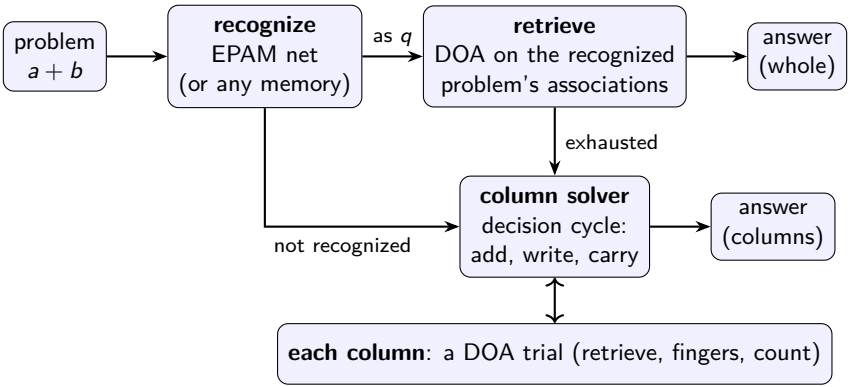
DOA (Siegler & Shrager 1984)

- Associations from problems to answers; sample, state if above a criterion; else fingers, then counting.
- Proved for every setting: retrieval used with probability $1 - (1 - R)^n$; counting errors independent of the associations.

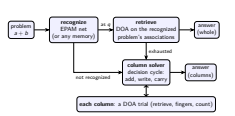


The two theories Robin has in mind were each formalized on their own, long before Chris's question. EPAM, Feigenbaum and Simon's model, recognizes a stimulus by sorting its code down a discrimination net; learning grows a test only where two codes are confused. Proved for every net: recognition depends only on the features tested, and learning never disturbs what's already learned. And the distributions-of-associations model that Bob Siegler and I proposed in 1984: a child samples answers in proportion to strength, states one that clears a confidence criterion, and otherwise falls back on fingers and counting. Proved for every setting: retrieval is used with probability one minus one minus R to the n, and counting errors don't depend on the associations at all, which was the chapter's nonintuitive prediction.

The application: three-digit addition



A demonstration, not a theory of arithmetic. In Lean, the composition is three definitions.



Chris composes them into a model of three-digit addition. A problem is recognized by an EPAM net, or by any memory. If it's recognized as a familiar problem q, the answer is retrieved from q's distribution of associations. If retrieval is exhausted, or nothing is recognized, a column solver takes over. It runs on the library's decision cycle, adding, writing and carrying, and each column's addition is itself a full single-digit trial from the strategy-choice model. In Lean the whole composition is three definitions. Again: arithmetic is the demonstration. It's a familiar task that needs several mechanisms, and two of the theories already existed.

The phenomenon, stated without a mechanism, and when it follows

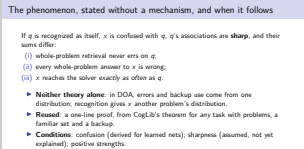
If q is recognized as itself, x is confused with q, q's associations are **sharp**, and their sums differ:

(i) whole-problem retrieval never errs on q;

(ii) every whole-problem answer to x is wrong;

(iii) x reaches the solver *exactly as often* as q.

- **Neither theory alone**: in DOA, errors and backup use come from one distribution; recognition gives x another problem's distribution.
- **Reused**: a one-line proof, from CogLib's theorem for *any* task with problems, a familiar set and a backup.
- **Conditions**: confusion (derived for learned nets); sharpness (assumed, not yet explained); positive strengths.

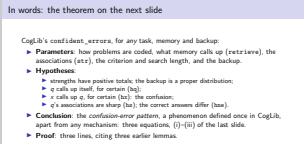


Chris and Robin first state the phenomenon without mentioning EPAM or DOA: a confused problem gets wrong answers from whole-problem retrieval, but uses its backup exactly as often as the problem it's confused with. Then they look for the conditions under which it follows. Suppose a familiar problem q is recognized as itself, a stranger x is confused with it, q's associations are sharp, meaning only q's correct sum clears the criterion, and the two sums differ. Then retrieval never errs on q, every whole-problem answer to x is wrong, and x goes to the solver exactly as often as q does. Neither theory says this alone: in strategy choice, a problem's errors and its backup use come from the same distribution, so they go together. Recognition breaks the tie by handing x another problem's distribution. The proof is one line, applying a library theorem that holds for any task with problems, a familiar set and a backup. And the conditions are explicit: confusion, which is derived for learned nets; sharpness, which is assumed and not yet explained; and positive strengths.

In words: the theorem on the next slide

CogLib's `confident_errors`, for *any* task, memory and backup:

- **Parameters**: how problems are coded, what memory calls up (`retrieve`), the associations (`str`), the criterion and search length, and the backup.
- **Hypotheses**:
  - strengths have positive totals; the backup is a proper distribution;
  - q calls up itself, for certain (`hq`);
  - x calls up q, for certain (`hx`): the confusion;
  - q's associations are sharp (`hs`); the correct answers differ (`hne`).
- **Conclusion**: the *confusion-error pattern*, a phenomenon defined once in CogLib, apart from any mechanism: three equations, (i)–(iii) of the last slide.
- **Proof**: three lines, citing three earlier lemmas.


In words: the theorem on the next slide

CogLib's `confident_errors`, for *any* task, memory and backup:

- **Parameters**: how problems are coded, what memory calls up (`retrieve`), the associations (`str`), the criterion and search length, and the backup.
- **Hypotheses**:
  - strengths have positive totals; the backup is a proper distribution;
  - q calls up itself, for certain (`hq`);
  - n calls up q, for certain (`hn`); the confusion;
  - q's associations are sharp (`hs`); the correct answers differ (`hne`).
- **Conclusion**: the *confusion-error pattern*, a phenomenon defined once in CogLib, apart from any mechanism: three equations, (i)–(iii) of the last slide.
- **Proof**: three lines, citing three earlier lemmas.

The next slide shows the actual Lean theorem, so here it is in words first. It's the library's version, which holds for any task, any memory and any backup. Its parameters are how problems are coded, what memory calls up for a code, the associations, the criterion and search length, and the backup. Its hypotheses: well-formedness of the strengths and the backup; the familiar problem calls up itself for certain; the stranger calls up the familiar problem for certain, which is the confusion; the familiar problem's associations are sharp; and the correct answers differ. The conclusion is the confusion-error pattern, which the library defines once, as a phenomenon, with no mechanism in it: the three equations from the previous slide. And the proof is three lines, each citing an earlier lemma.

The theorem, in Lean

```
theorem confident_errors (correct : P → A) (x q : P) (hT : ∀ y, total as (str y) ≠ 0)
    (hbx : mass (backup x) = 1) (hbq : mass (backup q) = 1)
    (hq : retrieve (code q) = pure (some (code q)))
    (hx : retrieve (code x) = pure (some (code q)))
    (hs : Sharp str cc (code q) (correct q)) (hne : correct x ≠ correct q) :
    Phenomena.ConfusionErrorPattern (recognizeRetrieve code retrieve str as cc n backup)
      (wholeWrong correct) isWhole isBackup x q := by
  refine ⟨confusion_gives_neighbours_answer code retrieve str as cc n backup
      correct q q (hT _) hq hs,
    wrong_eq_whole_of_sharp code retrieve str as cc n backup
      correct x _ _ (hT _) hx hs hne, ?_⟩
  rw [backup_prob_certain code retrieve str as cc n backup x _ hx hbx hT,
    backup_prob_certain code retrieve str as cc n backup q _ hq hbq hT]

def ConfusionErrorPattern (trial : P → Dist O) (wholeWrong : P → O → Bool)
    (isWhole isBackup : O → Bool) (x q : P) : Prop :=
  prob (trial q) (wholeWrong q) = 0 ∧
    prob (trial x) (wholeWrong x) = prob (trial x) isWhole ∧
    prob (trial x) isBackup = prob (trial q) isBackup
```

Verbatim from CogLib (`Process/RecognizeRetrieve`, `Phenomena/ConfusionErrors`). The multi-digit version is a one-line instance.

Composing Cognitive Theories in CogLean

The theorem, in Lean

The theorem, in Lean

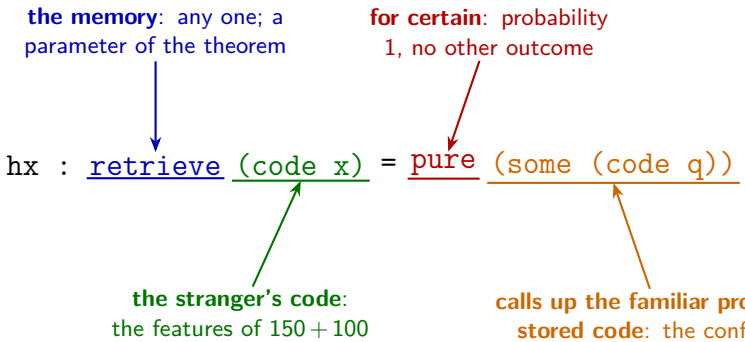
```
theorem confident_errors (correct : P → A) (q : P) (hT : ∀ y, total ac (stv y) ≠ 0)
    (hbu : mass (backup x) = 1) (hbq : mass (backup q) = 1)
    (hq : retrieves (code q) = pure (some (code q)))
    (hx : retrieves (code x) = pure (some (code q)))
    (hs : Sharp stv cc (code q) (correct q)) (hne : correct x ≠ correct q) :
    Phenomena.ConfusionErrorPattern (recognizeRetrieve code retrieve stv ac cc a backup)
      (wholeAnswer correct) (wholeBackup q q = hq)
  refine ⟨confusion_given_neighbours_answer code retrieve stv ac cc a backup
      correct q q (hT _) hq he,
    wrong_eq_whole_of_sharp code retrieve stv ac cc a backup
      correct q q (hT _) hq he hs, ?_⟩
  rw [backup_prob_certain code retrieve stv ac cc a backup q _ hx hbu hT,
    backup_prob_certain code retrieve stv ac cc a backup q _ hq hbq hT]

def ConfusionErrorPattern (trial : P → Dist O) (wholeWrong : P → O → Bool)
    (isWholeBackup : O → Bool) (x q : P) : Prop :=
  prob (trial q) (wholeWrong q) = 0 ∧
    prob (trial x) (wholeWrong x) = prob (trial x) isWholeBackup ∧
    prob (trial x) isWholeBackup = prob (trial q) isBackup
```

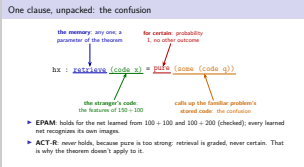
Verbatim from CogLib (Process/RecognizeRetrieve, Phenomena/ConfusionErrors). The multi-digit version is a one-line instance.

Here's the theorem itself, verbatim from the library. The first five lines are the hypotheses, each named: h T, the two backup hypotheses, h q, h x, h s and h n e. Then the conclusion: the trial has the confusion-error pattern. Then the proof. At the bottom is that pattern's definition, three equations joined by and, about observable outcomes only. The first clause is the lemma that a problem recognized as a sharp q gets only q's answer; the second is the lemma that all its whole-problem answers are then wrong; and the third rewrites both backup probabilities with the lemma that backup use after certain recognition depends only on what's recognized. Nothing in the statement mentions arithmetic, EPAM or ACT-R. The multi-digit model's theorem is a one-line instance of this one.

One clause, unpacked: the confusion



- **EPAM**: holds for the net learned from $100 + 100$ and $100 + 200$ (checked); every learned net recognizes its own images.
- **ACT-R**: *never* holds, because `pure` is too strong: retrieval is graded, never certain. That is why the theorem doesn't apply to it.



Let's unpack one clause, the confusion, h x. Read it left to right. Retrieve is the memory, and it's a parameter: the theorem holds for any memory you plug in. Code x is the stranger's feature code, here the features of one fifty plus one hundred. Pure means for certain: the distribution with a single outcome, at probability one. And some code q says the outcome is the familiar problem's stored code. So the clause says: this memory calls up the familiar problem, for certain, when given the stranger. For EPAM it holds: the net learned from one hundred plus one hundred and one hundred plus two hundred is checked to do exactly this. For ACT-R it never holds, and the culprit is the word pure: ACT-R's retrieval is graded and never certain. That single word is why the theorem applies to one memory and not the other.

Equal is not low

- The theorem gives *equal* solver use for x and q, not *low* solver use.
- The rate is $(1 - R_q)^n$, with R_q the chance one search of q's associations clears the criterion.
- If $R_q \geq r$, the confused problem reaches the solver with probability at most $(1 - r)^n$.
- With flat associations on q, the same theorem holds, and the errors are not confident.

Here $R_q = 45/46$, $n = 1$: the solver is reached $1/46$ of the time.

Equal is not low

- The theorem gives *equal* solver use for x and q, not *low* solver use.
- The rate is $(1 - R_q)^n$, with R_q the chance one search of q's associations clears the criterion.
- If $R_q \geq r$, the confused problem reaches the solver with probability at most $(1 - r)^n$.
- With flat associations on q, the same theorem holds, and the errors are not confident.

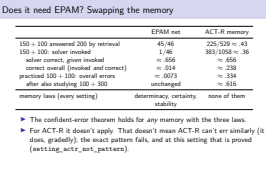
Here $R_q = 45/46$, $n = 1$: the solver is reached $1/46$ of the time.

One precision matters here. The theorem says the confused problem reaches the solver exactly as often as the familiar one. It doesn't say that's rare. The rate is one minus R q to the n, where R q is the chance that one search of q's associations clears the criterion; and if R q is at least r, solver use is at most one minus r to the n. Both are proved. So whether these errors are confident, fluent and unhesitating, is a separate, quantitative matter. With flat associations the same theorem holds, but the errors wouldn't be confident. In our setting R q is forty-five over forty-six, so the solver is reached once in forty-six.

Does it need EPAM? Swapping the memory

	EPAM net	ACT-R memory
$150 + 100$ answered 200 by retrieval	$45/46$	$225/529 \approx .43$
$150 + 100$: solver invoked	$1/46$	$383/1058 \approx .36$
solver correct, *given* invoked	$\approx .656$	$\approx .656$
correct overall (invoked *and* correct)	$\approx .014$	$\approx .238$
practiced $100 + 100$: overall errors	$\approx .0073$	$\approx .334$
after also studying $100 + 300$	unchanged	$\approx .616$
memory laws (every setting)	determinacy, certainty, stability	none of them

- The confident-error theorem holds for *any* memory with the three laws.
- For ACT-R it doesn't apply. That doesn't mean ACT-R can't err similarly (it does, gradedly); the exact pattern fails, and at this setting that is proved (`setting_actr_not_pattern`).



Chris's next question is whether any of this needs EPAM in particular. Because recognition is stated for any memory, we can swap EPAM's net for ACT-R's declarative memory and change nothing else. The confident-error pattern is proved for any memory with three laws: determinacy, certainty and stability. EPAM's net has all three at every setting; ACT-R's memory has none of them, also proved. So the theorem applies to EPAM and never to ACT-R. But be careful about what that means. It doesn't mean ACT-R can't make similar errors; at this setting it answers two hundred about forty-three percent of the time. What fails is the exact pattern: ACT-R's confusion is graded, and its solver use differs from the familiar problem's. A note on the solver rows: the solver is right about sixty-six percent of the time when it's invoked; the overall figure is that times how often it's invoked. It also predicts interference: studying a new problem raises the errors on a practiced one, which EPAM never does. All the numbers are exact, at one illustrative setting.

What Chris has now

- Not a simulation of one error, but a **reusable result**: sufficient conditions for the phenomenon, for every setting that meets them.
- It applies to a new model **without re-running** the old one, and to *any* memory with three laws.
- The conditions are where the science is:
  - Lean hasn't shown that children have sharp associations, or that a given learning history produces the confusion: separate theoretical or empirical questions;
  - the pattern by itself doesn't make backup use rare; *confident* needs a further, quantitative condition.

What Chris has now

- Not a simulation of one error, but a **reusable result**: sufficient conditions for the phenomenon, for every setting that meets them.
- It applies to a new model **without re-running** the old one, and to *any* memory with three laws.
- The conditions are where the science is:
  - Lean hasn't shown that children have sharp associations, or that a given learning history produces the confusion: separate theoretical or empirical questions;
  - the pattern by itself doesn't make backup use rare; confident needs a further, quantitative condition.

So what does Chris have now? Not a simulation that happened to produce one error, but a reusable result: a set of conditions sufficient for the phenomenon, proved for every setting that meets them. It applies to a new model without re-running the old one, and through the laws it applies to any memory with those three properties. And the conditions are where the science is. Lean hasn't shown that children have sharp associations, or that a particular learning history produces the confusion; those are separate questions, theoretical or empirical. And the pattern alone doesn't make backup use rare: calling the errors confident takes the further quantitative condition we just saw.

Part 2

Under the hood

How the composition in part 1 runs, and why its theorems transfer

Part 2

Under the hood

How the composition in part 1 runs, and why its theorems transfer

Part two is supporting detail: what's under the hood of the composition Chris used in part one, and why its theorems transfer.

What actually happens when retrieval fails

1. **Recognize**: the EPAM net sorts the problem's code to a stored problem, possibly the wrong one.
2. **Retrieve**: DOA searches that problem's associations for an answer above the criterion.
3. **Fail**: if the search is exhausted, the backup is invoked.
4. **Solve**: the backup runs column addition on CogLib's general problem-solving machinery.
5. **Each column**: units, then tens, then hundreds; each digit sum is a separate DOA trial (retrieve, else fingers, else count).
6. **Combine**: each trial's *distribution* of answers feeds the rest of the calculation, with carrying and place value.
7. **Result**: a probability distribution over complete multi-digit answers, errors included.

`Dist.bind` composes these steps: it carries *every* possible outcome, with its probability, through the steps that follow. Nothing is sampled, and nothing looks up the right answer: correct knowledge enters only as single-digit associations and the carry rule.

What actually happens when retrieval fails

1. **Recognize**: the EPAM net sorts the problem's code to a stored problem, possibly the wrong one.
2. **Retrieve**: DOA searches that problem's associations for an answer above the criterion.
3. **Fail**: if the search is exhausted, the backup is invoked.
4. **Solve**: the backup runs column addition on CogLib's general problem-solving machinery.
5. **Each column**: units, then tens, then hundreds; each digit sum is a separate DOA trial (retrieve, else fingers, else count).
6. **Combine**: each trial's distribution of answers feeds the rest of the calculation, with carrying and place value.
7. **Result**: a probability distribution over complete multi-digit answers, errors included.

`Dist.bind` composes these steps: it carries *every* possible outcome, with its probability, through the steps that follow. Nothing is sampled, and nothing looks up the right answer: correct knowledge enters only as single-digit associations and the carry rule.

First, an important detail: nothing in this model is handed the answer. Two hundred fifty for one fifty plus one hundred is assembled from single-digit trials and a carry rule, and it's never certain. So what actually happens when whole-problem retrieval fails? Step by step. The EPAM net sorts the problem's code to a stored problem, possibly the wrong one. DOA searches that problem's associations for an answer above the criterion. If the search is exhausted, the backup is invoked. The backup runs column addition on the library's general problem-solving machinery: units, then tens, then hundreds, and each column's digit sum is a separate single-digit DOA trial, with retrieval, fingers or counting. Each trial produces a distribution of possible answers, and that whole distribution feeds the rest of the calculation, with carrying and place value. The result is a probability distribution over complete multi-digit answers, errors included. The operation that does the composing is bind: it carries every possible outcome, with its probability, through the steps that follow. Nothing is sampled, and nothing looks up the right answer.

The general machinery, and its arithmetic instance

CogLib (`Process/ProblemSolving`), task-general:

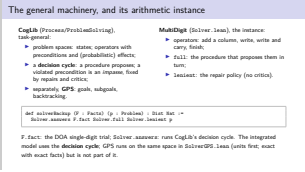
- problem spaces: states; operators with preconditions and (probabilistic) effects;
- a **decision cycle**: a procedure proposes; a violated precondition is an *impasse*, fixed by repairs and critics;
- separately, **GPS**: goals, subgoals, backtracking.

MultiDigit (`Solver.lean`), the instance:

- operators: add a column, write, write and carry, finish;
- `full`: the procedure that proposes them in turn;
- `lenient`: the repair policy (no critics).

```
def solverBackup (F : Facts) (p : Problem) : Dist Nat :=
  Solver.answers F.fact Solver.full Solver.lenient p
```

`F.fact`: the DOA single-digit trial; `Solver.answers`: runs CogLib's decision cycle. The integrated model uses the **decision cycle**; GPS runs on the same space in `SolverGPS.lean` (units first; exact with exact facts) but is not part of it.



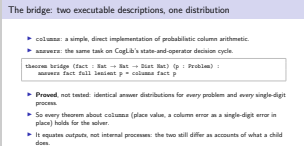
Here's the machinery underneath. The library's problem-solving module is task-general. It has a problem space, with states and operators that have preconditions and effects, and an effect can be a probability distribution. It has a decision cycle: a procedure proposes an operator; if its precondition is violated, that's an impasse, and repair heuristics and critics fix it. And, separately, it has GPS-style means-ends analysis, with goals, subgoals and backtracking. Multi-digit addition is one instance: operators to add a column, write, write and carry, and finish; a procedure, full, that proposes them in turn; and a repair policy, lenient. The backup is one call: answers, which runs the library's decision cycle, given the single-digit DOA trial, the procedure and the repair policy. One precision: the integrated model uses the decision cycle. GPS runs on the same problem space in a separate file, where it's proved to work units first and to be exact with exact facts, but it isn't part of the integrated model.

The bridge: two executable descriptions, one distribution

- `columns`: a simple, direct implementation of probabilistic column arithmetic.
- `answers`: the same task on CogLib's state-and-operator decision cycle.

```
theorem bridge (fact : Nat → Nat → Dist Nat) (p : Problem) :
    answers fact full lenient p = columns fact p
```

- **Proved**, not tested: identical answer distributions for *every* problem and *every* single-digit process.
- So every theorem about `columns` (place value, a column error as a single-digit error in place) holds for the solver.
- It equates *outputs*, not internal processes: the two still differ as accounts of what a child does.



Now the bridge. There are two executable descriptions of column addition. One, columns, is a simple direct implementation of probabilistic column arithmetic. The other, answers, does the same task on the library's general state-and-operator decision cycle. The bridge theorem says they produce identical answer distributions, for every problem and for every single-digit process you plug in. That's proved, not established by testing examples. Its payoff is that every theorem proved about the simple version, like place value, or a column error being a single-digit error moved to its place, holds for the solver too. But be precise about what it equates: outputs, not internal processes. As accounts of what a child does, the two are still different. This is the library idea in miniature: results transfer between independently defined mechanisms.

How theorems compose

- **The bridge** (previous slide): every column theorem transfers to the solver.
- **Associativity**: every arrow is a two-stage draw; draws regroup, so a component's theorem holds inside the whole.
- **Well-formedness, inherited**: the composed trial is a probability distribution, proved by chaining lemmas from three packages:
  - retrieval, a DOA trial, the column procedure, the bridge;
  - total mass 1, *and* nonnegative weights.

These are library facts, not psychology; they are what let results transfer.

How theorems compose

- **The bridge** (previous slide): every column theorem transfers to the solver.
- **Associativity**: every arrow is a two-stage draw; draws regroup, so a component's theorem holds inside the whole.
- **Well-formedness, inherited**: the composed trial is a probability distribution, proved by chaining lemmas from three packages:
  - retrieval, a DOA trial, the column procedure, the bridge;
  - total mass 1, *and* nonnegative weights.

These are library facts, not psychology; they are what let results transfer.

Three kinds of reuse make the composition deductive rather than just executable. A bridge theorem says the solver on the decision cycle answers exactly as a fixed column procedure does, for every problem and every single-digit process, so everything proved about the fixed procedure transfers. Associativity: every arrow is a two-stage draw of distributions, and draws regroup, so a theorem about a component holds where it sits inside the whole. And well-formedness is inherited: that the composed trial is a genuine probability distribution, total mass one and no negative weights, is proved by chaining lemmas from three packages. None of this is psychology. It's what lets the psychology transfer.

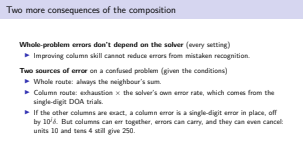
Two more consequences of the composition

Whole-problem errors don't depend on the solver (every setting)

- Improving column skill cannot reduce errors from mistaken recognition.

Two sources of error on a confused problem (given the conditions)

- Whole route: always the neighbour's sum.
- Column route: exhaustion $\times$ the solver's own error rate, which comes from the single-digit DOA trials.
- If the other columns are exact, a column error is a single-digit error in place, off by $10^j\delta$. But columns can err together, errors can carry, and they can even cancel: units 10 and tens 4 still give 250.



Two more consequences. First, whole-problem errors don't depend on the solver at all, at any setting. Psychologically that's a prediction about intervention: teaching column skill can't reduce errors that come from mistaken recognition; only learning that changes recognition or associations can. Second, a confused problem has two sources of error. The whole route always gives the neighbour's sum. The column route errs with probability equal to exhaustion times the solver's own error rate, which comes entirely from the single-digit trials. When only one column errs, the error is that single-digit error moved to its place. But that's a special case: columns can err together, errors propagate through carries, and they can even cancel. A units result of ten and a tens result of four still give two hundred fifty.

One memory for both levels

Column facts from the *same* memory as whole problems: each digit pair $d + e$ is coded as a problem, recognized, and retrieved in the same strength table; fingers and counting if that fails.

- **The old model is a special case**: a pair recognized as itself gives exactly the DOA fact (`memFact_self`), so every earlier theorem carries over.
- **Confident errors one level down**: a pair taken for another states the other's sum (`column_confident_errors`), with no new proof.
- **Noninterference needs a condition**: EPAM's practice stays undisturbed *given* that the problem's digit pairs were learned (`shared_learning_preserves_trial`). The old theorem held the solver fixed; here learning could change it.

One memory for both levels

Column facts from the same memory as whole problems: each digit pair $d + e$ is coded as a problem, recognized, and retrieved in the same strength table; fingers and counting if that fails.

- **The old model is a special case**: a pair recognized as itself gives exactly the DOA fact (`sameFact_self`), so every earlier theorem carries over.
- **Confident errors one level down**: a pair taken for another states the other's sum (`column_confident_errors`), with no new proof.
- **Noninterference needs a condition**: EPAM's practice stays undisturbed *given* that the problem's digit pairs were learned (`shared_learning_preserves_trial`). The old theorem held the solver fixed; here learning could change it.

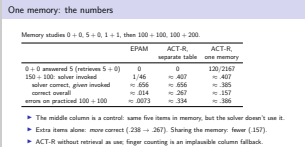
So far the solver's single-digit facts came from their own table, separate from the memory that recognizes whole problems. A child has one memory, so we removed the separation. Each digit pair, say five plus zero, is coded as a problem in the same code space, recognized by the same memory, and retrieved from the same strength table, with fingers and counting if retrieval fails. Three things follow. First, if a pair is recognized as itself, its fact is exactly the old DOA fact, so the old model is a special case and every earlier theorem carries over. Second, a pair taken for another pair gives confident errors one level down, by the same general theorem, with no new proof. Third, a caution: the old noninterference theorem held the solver fixed while the net learned. With one memory, learning could change the solver too, so noninterference needs a condition: it holds when the problem's digit pairs were themselves learned, and that we prove.

One memory: the numbers

Memory studies $0+0$, $5+0$, $1+1$, then $100+100$, $100+200$.

	EPAM	ACT-R, separate table	ACT-R, one memory
$0+0$ answered 5 (retrieves $5+0$)	0	0	120/2167
$150+100$: solver invoked	1/46	$\approx .407$	$\approx .407$
solver correct, *given* invoked	$\approx .656$	$\approx .656$	$\approx .385$
correct overall	$\approx .014$	$\approx .267$	$\approx .157$
errors on practiced $100+100$	$\approx .0073$	$\approx .334$	$\approx .386$

- The middle column is a control: same five items in memory, but the solver doesn't use it.
- Extra items alone: *more* correct ($.238 \rightarrow .267$). Sharing the memory: fewer (.157).
- ACT-R without retrieval as use; finger counting is an implausible column fallback.


One memory: the numbers

Memory studies $0+0$, $5+0$, $1+1$, then $100+100$, $100+200$.

	EPAM	ACT-R, separate table	ACT-R, one memory
$0+0$ answered 5 (retrieves $5+0$)	0	0	126/2167
$150+100$: solver invoked	1/46	$\approx .407$	$\approx .407$
solver correct, *given* invoked	$\approx .656$	$\approx .656$	$\approx .385$
correct overall	$\approx .014$	$\approx .267$	$\approx .157$
errors on practiced $100+100$	$\approx .0073$	$\approx .334$	$\approx .386$

- The middle column is a control: same five items in memory, but the solver doesn't use it.
- Extra items alone: *more* correct (.238 → .267). Sharing the memory: fewer (.157).
- ACT-R without retrieval as use; finger counting is an implausible column fallback.

Here are the numbers, with the memory holding three digit pairs and the two problems. EPAM recognizes every pair as itself, so nothing changes from before. ACT-R confuses pairs: zero plus zero sometimes retrieves five plus zero and is answered five. Two things changed at once for ACT-R, the memory holds more items and the solver uses it, so the middle column is a control: the same five items in memory, but the solver keeps its separate table. The extra items alone actually raise correct answers a little, from about twenty-four to twenty-seven percent, because more problems reach an accurate solver. It's the solver's use of the memory that brings them down to about sixteen: the solver's accuracy when invoked drops from sixty-six percent to under forty. Two caveats: this is ACT-R without retrieval changing memory, and finger counting is an implausible fallback for someone doing three-digit addition.

Part 3

What this shows, and what it doesn't

Checks, kinds of results, limits

Finally, what all this shows and doesn't: how results are labeled, how we checked the formalization, where it sits, and its limits.

What each result rests on

Kind	Examples
Library facts	the bridge; the composed trial is a probability distribution
Compositional deductions (every setting)	whole-problem errors independent of the solver; solver use is $(1 - R_q)^n$
Conditional	confident errors; two sources of error; any memory with three laws
At chosen settings	$45/46$, $1/46$, $\approx .656$, $\approx .014$, the ACT-R column

None of these is yet an empirical finding about people. The numbers are illustrations, not fits.

Composing Cognitive Theories in CogLean

What each result rests on

What each result rests on

Kind	Examples
Library facts	the bridge; the composed trial is a probability distribution
Compositional deductions (every setting)	whole-problem errors independent of the solver; solver use is $(1 - R_d)^n$
Conditional	confident errors; two sources of error; any memory with three laws
At chosen settings	45/46, 1/46, $\approx .656$, $\approx .014$, the ACT-R column

None of these is yet an empirical finding about people. The numbers are illustrations, not fits.

For credibility, it's essential to say what each result rests on. Some are library facts, like the bridge and well-formedness: needed, but not psychology. Some are compositional deductions that hold at every setting, like solver independence. Some are conditional, like confident errors, and the conditions are where the science is: the proofs say exactly which premise a failed test would implicate. And some are just numbers at chosen settings. None of these is yet an empirical finding about people, and the numbers are illustrations, not fits to children's data.

Checking the formalization: a blind simulation

- A prose specification, no Lean; preregistered checks; a fresh agent wrote a simulator from the prose alone.
- **Agreement**: every check whose hypothesis arose passed; 243 frequencies matched Lean exactly.
- **Found what proof didn't**:
  - five theorems' hypotheses were *unrealized* in the tested learning histories (not shown unreachable);
  - a gap at zero total strength;
  - a predicted learning sequence depends on an unstated assumption: a new problem's initial associations.

Checking the formalization: a blind simulation

- A prose specification, no Lean; preregistered checks; a fresh agent wrote a simulator from the prose alone.
- **Agreement**: every check whose hypothesis arose passed; 243 frequencies matched Lean exactly.
- **Found what proof didn't**:
  - five theorems' hypotheses were *unrealized* in the tested learning histories (not shown unreachable);
  - a gap at zero total strength;
  - a predicted learning sequence depends on an unstated assumption: a new problem's initial associations.

Lean checks that the theorems follow from the definitions, not that the definitions say what the prose model says. So we checked that link independently. A prose specification with no Lean in it, checks preregistered before any simulator existed, and a fresh AI agent that wrote a simulator from the prose alone. Every check whose hypothesis arose passed, and two hundred forty-three outcome frequencies matched Lean's exact values. And it found things the proofs didn't. Five theorems had hypotheses that never arose in the learning histories we tested; that doesn't show they're unreachable, only that the theorems said little about those nets. There's a gap at zero total strength. And a predicted learning sequence turns out to depend on an assumption the model never stated: the initial associations of a newly discriminated problem.

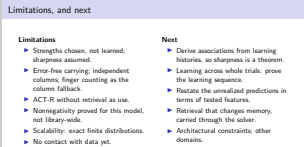
Limitations, and next

Limitations

- Strengths chosen, not learned; sharpness assumed.
- Error-free carrying; independent columns; finger counting as the column fallback.
- ACT-R without retrieval as use.
- Nonnegativity proved for this model, not library-wide.
- Scalability: exact finite distributions.
- No contact with data yet.

Next

- Derive associations from learning histories, so sharpness is a theorem.
- Learning across whole trials: prove the learning sequence.
- Restate the unrealized predictions in terms of tested features.
- Retrieval that changes memory, carried through the solver.
- Architectural constraints; other domains.


Limitations, and next

Limitations

- Strengths chosen, not learned; sharpness assumed.
- Error-free carrying; independent columns; finger counting as the column fallback.
- ACT-R without retrieval as use.
- Nonnegativity proved for this model, not library-wide.
- Scalability: exact finite distributions.
- No contact with data yet.

Next

- Derive associations from learning histories, so sharpness is a theorem.
- Learning across whole trials: prove the learning sequence.
- Restate the unrealized predictions in terms of tested features.
- Retrieval that changes memory, carried through the solver.
- Architectural constraints; other domains.

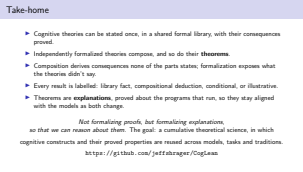
The limitations are real. The associative strengths are chosen, not learned, and sharpness, the key premise of confident errors, is assumed. Carrying is error-free and columns are independent, and the columns fall back on finger counting, which no one doing three-digit addition would do. ACT-R is represented by its retrieval rule only: retrieving something doesn't change its memory, so all the ACT-R numbers are for that version. Nonnegative weights are proved for this model but not yet across the library. Exact finite distributions raise scalability questions. And there's no contact with data yet. Next: derive the associations from learning histories, so that sharpness becomes a theorem instead of an assumption; add learning across whole trials, so the predicted learning sequence can be proved; restate the predictions whose hypotheses weren't realized; let retrieval change memory, carrying the memory's state through the solver; and move on to architectural constraints and other domains.

Take-home

- Cognitive theories can be stated once, in a shared formal library, with their consequences proved.
- Independently formalized theories compose, and so do their **theorems**.
- Composition derives consequences none of the parts states; formalization exposes what the theories didn't say.
- Every result is labelled: library fact, compositional deduction, conditional, or illustrative.
- Theorems are **explanations**, proved about the programs that run, so they stay aligned with the models as both change.

Not formalizing proofs, but formalizing explanations, so that we can reason about them. The goal: a cumulative theoretical science, in which cognitive constructs and their proved properties are reused across models, tasks and traditions.

`https://github.com/jeffshrager/CogLean`



To sum up. Cognitive theories can be stated once, in a shared formal library, with their consequences proved rather than simulated. Independently formalized theories compose, and so do their theorems. Composition derives consequences that none of the parts states, and formalization exposes things the theories never said. And every result is labelled: a library fact, a compositional deduction, a conditional result, or an illustration. The goal is a cumulative theoretical science, in which cognitive constructs and their proved properties are reused across models, tasks and research traditions, the way mathematical results are reused in Mathlib. One example doesn't establish that. It shows that the approach can produce results of the right kind. The point is not merely to run cognitive models, but to reason about them. CogLean isn't primarily about formalizing proofs. It's about formalizing explanations so that we can reason about them, and using Lean to keep those explanations rigorously aligned with the computational models they explain.